Runtime Trust Kit

A framework-neutral starter kit for making an AI workflow's runtime behaviour inspectable, bounded, and repairable after deployment.

The core contract

The kit is intentionally not a prompt library, an agent framework, or a claim that a model can reliably grade its own work. It is a compact example of the controls that should survive a model swap.

01	Name the action and its bounded authority.
02	Collect evidence as observable signals, not a self-reported confidence score.
03	Choose an explicit runtime outcome: proceed, abstain, or send to a maintainer.
04	Keep human disposition, accountable rationale, execution acknowledgement, and independently observed effect distinct.
05	Keep a trace that can become a fixture, an evaluation, and eventually a regression test.

The improvement loop

bad answer → inspectable trace → fixture → evaluation → regression test → policy / code / model change

That is the practical connection to open source: a failure becomes something a community can inspect, reproduce, improve, and share.

What is included

Use these pieces together as a small, model-independent runtime trust contract.

Policy

Names the allowed action, evidence requirements, escalation rule, and expected proof of completion.

Fixture

A concrete, shareable issue with stale and conflicting evidence, so a claim can be tested rather than admired.

Evaluation

Sets the pass condition: escalate to a maintainer; do not invent an unconditional compatibility claim.

Traces

Separates runtime outcome, human disposition, accountable rationale, execution acknowledgement, and independently observed effect.

Verifier

A dependency-free contract check that makes the example runnable with ordinary Node.js.

Adapt it

1. Copy the policy and replace the example action with your real workflow action.
2. List signals your application can actually observe. Do not invent a magic confidence number.
3. Set the smallest authority that is useful. Make broader actions require human approval or a separate policy.
4. Specify the downstream state that proves completion. A successful API call is an execution acknowledgement, not proof that the intended effect occurred.
5. Preserve runtime outcome, human disposition, accountable rationale, execution acknowledgement, and independently observed effect as separate fields.
6. Turn your next notable production miss into a redacted fixture and an evaluation case.

Boundaries

This kit is a design and testing pattern, not a security certification. It does not prevent prompt injection, compromised dependencies, or a malicious tool. It makes their effects more explicit and gives a maintainer a place to add controls.

Start with the threat model and adapt it to your system. The source files, examples, and dependency-free verifier remain available at rbx-labs.io/runtime-trust-kit/.